

Paytm Scam Shield

+ Smart Pay Assistant (Grok Bot)

Complete Hackathon Build Guide

Idea | Prompts | API Samples | React Structure | Demo Script

Goal: Use this document as a single reference to build a ready-to-demo application for a hackathon - quickly, clearly, and impressively.

One-liner

"Suspicious UPI / payment message ko Grok se check karo - risk score, reason, aur safe next step, Hinglish me."

Version 1.0 | Hackathon Ready Pack

Table of Contents

- 1. Product Overview
- 2. Problem & Why Paytm
- 3. Features (MVP + Stretch)
- 4. User Flow
- 5. Tech Stack
- 6. Project File Structure
- 7. Grok System Prompt (Copy-Paste)
- 8. User Message Template
- 9. Fake Analyze API Samples (3 Scenarios)
- 10. Backend API Spec
- 11. Frontend Component Guide
- 12. Starter Code Snippets
- 13. Build Timeline (Hackathon Day)
- 14. 3-Minute Demo Script
- 15. Pitch Slides Outline
- 16. Team Split
- 17. Checklist - Demo Ready
- 18. Stretch Ideas & Closing Pitch

1. Product Overview

Paytm Scam Shield is an AI safety assistant powered by Grok. Before a user pays via UPI or follows a payment-related SMS/chat, the app analyzes the text and returns: risk level (Safe / Caution / High Risk), score (0-100), short reasons, recommended action, and a Hinglish summary.

This is perfect for a hackathon because: (1) India UPI scam problem is real, (2) Grok reasoning is visible and impressive, (3) UI can be mocked without real bank/UPI settlement, (4) demo is live and interactive in under 3 minutes.

Important:

Do NOT process real money in the hackathon demo. Use mock pay screens only. Always label the product as advisory / educational for safety.

2. Problem & Why Paytm

Problem

- UPI fraud and social engineering are rising (fake KYC, urgency, phishing links).
- Users often pay under pressure without a second check.
- Support FAQs alone are not enough at the moment of payment.
- Hinglish users need simple, calm guidance - not long English policies.

Why this fits Paytm

- Paytm is trusted for payments - adding a safety layer strengthens trust.
- Natural place in the pay flow: Check -> Score -> Act -> Pay (if safe).
- Can later connect to SMS, chat, QR, and merchant flows.
- Differentiates Paytm as 'smart + safe', not only 'fast pay'.

3. Features (MVP + Stretch)

Must-have (MVP)

1. Chat / paste UI for SMS, UPI request, or chat text
2. Risk score: Safe / Caution / High Risk
3. 3 short reasons + recommended action
4. Hinglish summary for the user
5. Mock Confirm Pay screen (only when Safe)
6. 3 sample scenario buttons for live demo

Nice-to-have (if time left)

- Voice input (Hinglish transcript -> analyze)
- Split-bill suggestion from bill text
- Smart Spend tip on fake transaction history
- Report scam button (stores local log)

Out of scope

- Real UPI settlement / bank APIs
- Full KYC flows
- Production auth / payments compliance

4. User Flow

```
Home
-> CTA: Check before you pay
Analyze Screen
-> Paste SMS / UPI text OR click Sample Scenario
-> Loading: Grok check kar raha hai...
-> Risk Card (score + reasons + hindi_summary)
-> Actions:
    High Risk / Caution -> Block / Verify First
    Safe -> Continue to Pay (MockPay success screen)
```

Demo sample inputs

- High Risk: Fake KYC link + UPI PIN ask + bit.ly link
- Caution: Relative emergency + new UPI ID + urgency
- Safe: Official electricity biller + amount + due date

5. Tech Stack

Layer	Recommended choice
Frontend	React + Vite (or Next.js)
Styling	Plain CSS / Tailwind (Paytm-like blue)
AI	Grok API (xAI)
Backend	Node Express OR Vite proxy + serverless function
Storage	None required; local JSON mocks are enough
Fallback	Hardcoded mockResponses if API key/network fails

Environment variable: GROK_API_KEY=your_key_here (never commit real keys).

6. Project File Structure

```
paytm-scam-shield/
package.json
.env.example          # GROK_API_KEY=
public/
src/
  main.jsx
  App.jsx
  index.css
  pages/
    Home.jsx
    Analyze.jsx
    MockPay.jsx
  components/
    Navbar.jsx
    ChatInput.jsx
    RiskCard.jsx
    ReasonList.jsx
    ActionButtons.jsx
    SampleScenarios.jsx
  services/
    analyzeApi.js
    mockResponses.js
  utils/
    riskStyles.js
server/ (optional)
  index.js
  routes/analyze.js
  prompts/scamShield.js
```

7. Grok System Prompt (Copy-Paste)

Save this exactly in server/prompts/scamShield.js or send as system role message.

You are "Paytm Scam Shield", a safety assistant for UPI and digital payments in India.

Your job:

- Analyze user-provided payment text (SMS, chat, UPI request, link, voice transcript).
- Detect scam patterns: fake KYC, urgency, unknown UPI collect requests, phishing links, lottery/refund fraud, impersonation (bank/Paytm/police), prepaid verification of Rs 1/Rs 2, QR traps.
- Be practical, calm, and clear. Prefer Hinglish for user-facing summaries.
- NEVER encourage sending money to risky parties.
- Do NOT invent bank policies. If unsure, mark Caution and ask for more details.
- This is advisory only; final payment decision is user's.

Output STRICTLY valid JSON only (no markdown, no extra text):

```
{
  "risk": "Safe" | "Caution" | "High Risk",
  "score": 0-100,
  "reasons": ["short reason 1", "short reason 2", "short reason 3"],
  "recommended_action": "one clear next step",
  "hindi_summary": "2-3 short Hinglish/Hindi lines for the user",
  "red_flags": ["flag1", "flag2"],
```

```
"safe_to_proceed": true | false,  
"suggested_ui": {  
  "primary_button": "Block / Verify / Continue to Pay",  
  "secondary_button": "Ask more / Report"  
}  
}
```

Scoring guide:

- 0-30 Safe
- 31-69 Caution
- 70-100 High Risk

If input is incomplete, still return JSON with Caution and ask what is missing inside hindi_summary.

8. User Message Template

Analyze this payment-related text for scam risk.

Text:

```
""  
{{USER_INPUT}}  
""
```

Context (optional):

- App: Paytm-like demo
- Locale: India
- Language preference: Hinglish

9. Fake Analyze API Samples (3 Scenarios)

Use these for offline demo and as fallback when Grok API is unavailable. Keep response shape identical to live Grok output.

Scenario A - High Risk (fake KYC link)

Request text:

```
Aapka Paytm KYC pending hai. 24 ghante me band ho jayega. Is link pe  
Rs 1 verify kare: https://bit.ly/paytm-kyc-verify aur UPI PIN dale.
```

Response JSON:

```
{  
  "risk": "High Risk",  
  "score": 92,  
  "reasons": [  
    "Urgent KYC threat + short link is a common scam pattern",  
    "Asks for UPI PIN on an external link",  
    "Rs 1 verification is often used to steal account access"  
  ],  
  "recommended_action": "Do not click the link. Do not enter UPI PIN.  
Open official Paytm app only.",  
  "hindi_summary": "Ye message scam lag raha hai. Link mat kholo aur  
UPI PIN kahin mat dalo.",  
  "red_flags": ["urgency", "shortened_link", "upi_pin_request"],  
  "safe_to_proceed": false,  
  "suggested_ui": {  
    "primary_button": "Block & Ignore",  
    "secondary_button": "Report Scam"  
  }  
}
```

Scenario B - Caution (relative emergency)

Request text:

```
Bhai emergency hai, phone chori ho gaya. Is naye number se baat kar  
raha hoon. Turant 5000 is UPI pe bhej do: emergency.help@oksbi
```

Response JSON:

```
{  
  "risk": "Caution",  
  "score": 64,  
  "reasons": [  
    "Urgent money request from a new/unknown UPI handle",  
    "Lost phone story is a common social-engineering pattern",  
    "No independent verification yet"  
  ],  
  "recommended_action": "Call on previously saved number. Confirm  
identity before paying.",  
  "hindi_summary": "Pehle verify karo. Purane number pe call karke  
confirm kiye bina 5000 mat bhejo.",  
  "red_flags": ["urgency", "new_upi_id", "identity_not_verified"],  
  "safe_to_proceed": false,  
  "suggested_ui": {  
    "primary_button": "Verify First",  
    "secondary_button": "Continue Anyway"  
  }  
}
```

```
}
```

Scenario C - Safe (electricity bill)

Request text:

Electricity bill due: Rs 842 for consumer no. thr-22019. Pay via official biller 'State Electricity Board' in Paytm. Due date 5 Oct.

Response JSON:

```
{
  "risk": "Safe",
  "score": 18,
  "reasons": [
    "Looks like a normal utility bill payment request",
    "Mentions official biller flow instead of personal UPI",
    "No PIN, KYC link, or urgency scam language detected"
  ],
  "recommended_action": "Pay only through official biller. Confirm consumer number and amount.",
  "hindi_summary": "Ye normal bill payment lagta hai. Official biller se pay karo, amount aur consumer number check kar lena.",
  "red_flags": [],
  "safe_to_proceed": true,
  "suggested_ui": {
    "primary_button": "Continue to Pay",
    "secondary_button": "Ask Shield"
  }
}
```

10. Backend API Spec

Endpoint

POST /api/analyze
Content-Type: application/json

```
{ "text": "paste message here" }
```

Success response

Same JSON schema as Section 7/9 (risk, score, reasons, recommended_action, hindi_summary, red_flags, safe_to_proceed, suggested_ui).

Logic

1. Validate text is non-empty (else 400).
2. Call Grok with system prompt + user template.
3. Parse model output as JSON (strip markdown fences if model adds them).
4. On parse/network failure: keyword match to mock scenario OR return Caution fallback.
5. Never return free-form prose from API - always JSON.

Simple keyword fallback (offline demo)

- If text contains 'KYC' or 'bit.ly' or 'UPI PIN' -> High Risk mock
- If text contains 'emergency' and '@' -> Caution mock
- If text contains 'electricity' or 'biller' -> Safe mock
- Else -> Caution fallback asking for more detail

11. Frontend Component Guide

File	Responsibility
Home.jsx	Problem statement + Start Check CTA
Analyze.jsx	Input -> API -> RiskCard + actions
ChatInput.jsx	Textarea + Analyze button
SampleScenarios.jsx	1-click fill for 3 demos
RiskCard.jsx	Score, risk color, hindi_summary
ReasonList.jsx	Bullet reasons + red flags
ActionButtons.jsx	Primary/secondary from suggested_ui
MockPay.jsx	Fake success only when Safe
analyzeApi.js	fetch /api/analyze + error handling
mockResponses.js	3 scenario JSON objects
riskStyles.js	green / amber / red mapping

UI states

- Empty - show samples and placeholder
- Loading - 'Grok check kar raha hai...'
- Result - colored RiskCard
- Error - toast + auto fallback to mock

Risk colors

- Safe: green (#16a34a)
- Caution: amber (#d97706)
- High Risk: red (#dc2626)
- Brand accent: Paytm-like blue (#00baf2 / #002970)

12. Starter Code Snippets

mockResponses.js (core)

```
export const SCENARIOS = {
  highRisk: { id: 'highRisk', label: 'Fake KYC SMS',
    text: 'Aapka Paytm KYC pending hai...',
    response: { risk: 'High Risk', score: 92, /* ... */ } },
  caution: { id: 'caution', label: 'Emergency UPI request',
    text: 'Bhai emergency hai...',
    response: { risk: 'Caution', score: 64, /* ... */ } },
  safe: { id: 'safe', label: 'Electricity bill',
    text: 'Electricity bill due: Rs 842...',
    response: { risk: 'Safe', score: 18, /* ... */ } },
};

export function pickMockByText(text = '') {
  const t = text.toLowerCase();
  if (t.includes('kyc') || t.includes('bit.ly'))
    return SCENARIOS.highRisk.response;
  if (t.includes('emergency')) return SCENARIOS.caution.response;
}
```

```
if (t.includes('electricity') || t.includes('biller'))
  return SCENARIOS.safe.response;
return { risk: 'Caution', score: 50, reasons: ['Need more context'],
  recommended_action: 'Share full message', hindi_summary:
  'Thoda aur detail bhejo.', red_flags: [], safe_to_proceed: false,
  suggested_ui: { primary_button: 'Verify First',
    secondary_button: 'Edit Text' } };
```

analyzeApi.js

```
import { pickMockByText } from './mockResponses';

export async function analyzeText(text, { useMock = false } = {}) {
  if (!text?.trim()) throw new Error('Please paste a message');
  if (useMock) return pickMockByText(text);
  try {
    const res = await fetch('/api/analyze', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text }),
    });
    if (!res.ok) throw new Error('API failed');
    return await res.json();
  } catch (e) {
    console.warn('Falling back to mock', e);
    return pickMockByText(text);
  }
}
```

Analyze.jsx flow (pseudo)

```

1. const [text, setText] = useState('')
2. const [result, setResult] = useState(null)
3. const [loading, setLoading] = useState(false)
4. onAnalyze:
    setLoading(true)
    result = await analyzeText(text)
    setResult(result); setLoading(false)
5. Render ChatInput + SampleScenarios + RiskCard
6. If result.safe_to_proceed && Continue clicked
    -> navigate('/mock-pay')

```

Grok API call sketch (server)

```

// pseudo - adapt to current xAI Grok HTTP API
POST https://api.x.ai/v1/chat/completions
Authorization: Bearer GROK_API_KEY
{
  "model": "grok-2-latest",
  "temperature": 0.2,
  "messages": [
    { "role": "system", "content": SYSTEM_PROMPT },
    { "role": "user", "content": userTemplate(text) }
  ]
}
// Then JSON.parse(content) after cleaning ```json fences

```

13. Build Timeline (Hackathon Day)

Time	Task
0-1 hr	Scaffold React app, theme, Home + Navbar
1-2 hr	Analyze page + mocks + RiskCard working offline
2-4 hr	Backend /api/analyze + Grok prompt integration
4-5 hr	MockPay, sample buttons, polish colors/copy
5-6 hr	Error fallback, demo rehearsal (3 scenarios)
Last hr	Pitch deck + 3-min script practice

14. 3-Minute Demo Script

Time	Say / Show
0:00-0:20	Problem: UPI scams rising; people pay in panic
0:20-0:50	Solution: Check before pay with Grok - Hinglish
0:50-1:40	LIVE: paste fake KYC SMS -> High Risk card
1:40-2:20	LIVE: electricity bill -> Safe -> Mock Pay
2:20-2:45	Impact: fewer frauds, more trust for Paytm
2:45-3:00	Next: voice, SMS auto-detect, family alerts

Closing line:

Paytm pe payment se pehle ek smart second - Scam Shield by Grok.

15. Pitch Slides Outline

1. Title - Paytm Scam Shield + Grok
2. Problem - UPI fraud, fake KYC, urgency scams
3. Solution - Check -> Score -> Act
4. Live demo / screenshots
5. How it works - User -> App -> Grok -> Risk card
6. Why Paytm - trust, payments graph, Hinglish users
7. Roadmap - SMS auto-detect, voice, family alerts

16. Team Split (4 people)

Who	Owns
A	UI: Home, Analyze, RiskCard, MockPay
B	API: /analyze + Grok integration + fallback
C	Prompt tuning + 3 demo scenarios + edge cases
D	Pitch deck + demo script + fake data QA

17. Checklist - Demo Ready

- ☐ App opens on laptop without errors
- ☐ Sample High Risk scenario works offline
- ☐ Sample Caution scenario works
- ☐ Sample Safe scenario -> MockPay success
- ☐ Loading state visible
- ☐ API failure falls back to mocks (no blank screen)
- ☐ Hinglish summary readable on RiskCard
- ☐ Brand colors look Paytm-like (blue theme)
- ☐ No real API keys shown on screen/slides
- ☐ 3-minute script rehearsed twice
- ☐ Backup: screenshots/GIF if live Wi-Fi fails

18. Stretch Ideas & Closing Pitch

Stretch

- Voice: '500 bhejna hai Rahul ko - pehle check karo'
- Smart Spend Coach tab on fake transactions
- Family Money Mode - limits + simple summary
- Merchant helper for kirana QR explainers

Judges - 3 impress points

1. Real India problem (UPI scams)
2. Clear AI use (Grok reasoning + Hinglish)
3. Paytm product fit (safety before payment)

Final note:

First make the offline mock demo flawless. Then connect Grok. A beautiful fallback demo beats a broken live API call.

End of Guide - Go build. All the best for the hackathon!